DONANIM VE GÖMÜLÜ SİSTEMLER

ESP32 SORULAR VE CEVAPLAR

Anlaşılır, Uygulama Odaklı ve Görsel Rehber

GPIO • Bellek • I2C • SPI • UART
ADC/DAC • Wi-Fi • Bluetooth • ESP-NOW
Sensör Entegrasyonu



HAZIRLAYAN

Osman Melih Gümüş

Temmuz 2026

Bu Rehber Nasıl Kullanılmalı?

Bu belge, ESP32 hakkında sık sorulan fakat çoğu zaman birbirinden kopuk anlatılan konuları tek bir öğrenme sırasına yerleştirir. Amaç, yalnızca "hangi pin kullanılır?" sorusunu cevaplamak değil; seçimin arkasındaki mantığı da kısa ve anlaşılır biçimde göstermektir.

Kapsam Notu

Bu rehber ağırlıklı olarak **klasik ESP32 / ESP32-WROOM-32 modülü kullanılan tipik 38 pinli geliştirme kartını** esas alır. ESP32-S2, S3, C3, C6 ve diğer aile üyelerinde pinler, Bluetooth desteği, DAC kanalları ve bazı çevre birimleri farklı olabilir. Kullandığınız kartın üzerindeki modül kodunu ve kart pin şemasını mutlaka kontrol edin.[1]

Önerilen Okuma Sırası

1. Önce **bellek, boot ve GPIO** bölümlerini okuyun.
2. Ardından **ADC/DAC/PWM** ile fiziksel sinyallerin nasıl işlendiğini öğrenin.
3. Sonra **I2C, SPI ve UART** arasındaki farkları karşılaştırın.
4. En son **Wi-Fi, Bluetooth ve ESP-NOW** kararını verin.
5. Bir sensör bağlamadan önce Sensörle Çalışma Akışı kontrol listesini uygulayın.

İçindekiler

Bu Rehber Nasıl Kullanılmalı? i

1 ESP32'yi Doğru Çerçevede Tanımak 1
1.1 Bir Mikrodenetleyici Projesinde Temel Akış . . . 1

2 Bellek, Kodun Çalışması ve Boot Süreci 1
2.1 Flash ve SRAM . . . 1
2.2 Stack, Heap ve Statik Bellek . . . 2
2.3 Boot, Reset ve Upload . . . 2

3 GPIO Mantığı ve Güvenli Pin Seçimi 3
3.1 GPIO Giriş ve Çıkış Mantığı . . . 3
3.2 GPIO Matrix Nedir? . . . 4
3.3 GPIO 6–11 Neden Kullanılmamalı? . . . 5
3.4 Pratik Pin Seçim Tablosu . . . 5
3.5 Pin Seçme Algoritması . . . 6

4 Dijital ve Analog Dünya: ADC, DAC ve PWM 6
4.1 Üç Kavramın Farkı . . . 6
4.2 ADC Nasıl Çalışır? . . . 6
4.3 DAC Nasıl Çalışır? . . . 7
4.4 PWM Neden Analog Gibi Görünür? . . . 7
4.5 LDR ile Otomatik Aydınlatma Mantığı . . . 8

5 Kablolu Haberleşme: I2C, SPI ve UART 8
5.1 Genel Karşılaştırma . . . 8
5.2 I2C . . . 9
5.3 SPI . . . 9
5.4 UART . . . 10
5.5 Hangi Protokolü Seçmeliyim? . . . 10

6 Kablosuz Haberleşme: Wi-Fi, Bluetooth ve ESP-NOW 10
6.1 Wi-Fi 802.11 b/g/n . . . 11
6.2 Bluetooth Classic ve BLE . . . 11
6.3 Zaman Paylaşımı ve RF Coexistence . . . 11
6.4 ESP-NOW . . . 12
6.5 Hangi Kablosuz Yöntem Seçilmeli? . . . 13

7 Sensörlerle Çalışma Rehberi 13
7.1 Sensörü Bağlamadan Önce Sorulacak Sorular . . . 13
7.2 Modül ile Çıplak Sensör Farkı . . . 14
7.3 Veri Okuma Döngüsü Nasıl Tasarlanmalı? . . . 14
7.4 Örnek: I2C Basınç Sensörü + ESP-NOW . . . 15

8 Sık Sorulan Sorular ve Arıza Giderme 15
8.1 Kart Açılmıyor veya Boot Loop'a Giriyor . . . 15
8.2 Kod Yüklenmiyor . . . 15
8.3 Wi-Fi Açılınca Analog Değer Bozuluyor . . . 15
8.4 I2C Sensörü Bulunamıyor . . . 16
8.5 ESP-NOW Paketleri Bazen Gelmiyor . . . 16

8.6 LED Çok Kısık Yanıyor 16
8.7 GPIO34'e LED Bağladım, Neden Yanmıyor? 16
8.8 Serial Monitor'da Anlamsız Karakterler Var 16

9 Hızlı Başvuru Sayfası **17**
9.1 Bir Bakışta Pin Seçimi 17
9.2 Bir Bakışta Haberleşme Seçimi 17
9.3 Proje Başlatma Kontrol Listesi 17

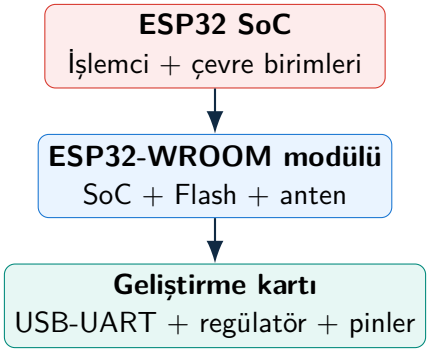
10 Yardımcı Kaynaklar ve Kaynakça **18**

1 ESP32'yi Doğru Çerçevede Tanımak

ESP32 tam olarak nedir?

ESP32; işlemci, RAM, kablosuz haberleşme birimleri ve çok sayıda çevre birimini tek çipte birleştiren bir **mikrodenetleyici ailesidir**. Klasik ESP32-WROOM-32 modülünde çift çekirdekli işlemci, 520 KB SRAM, tipik 4 MB harici SPI Flash, 2.4 GHz Wi-Fi ve Bluetooth Classic/BLE bulunur.[1]

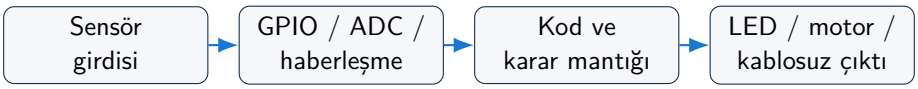
Bir geliştirme kartına baktığınızda aslında üç katman görürsünüz:



Kısa İpucu

Arduino IDE'de "ESP32 Dev Module" seçmeniz, kartın tamamının aynı olduğu anlamına gelmez. Aynı isimle satılan kartlarda farklı USB-UART çipleri, farklı regülatörler veya farklı modüller bulunabilir.

1.1 Bir Mikrodenetleyici Projesinde Temel Akış



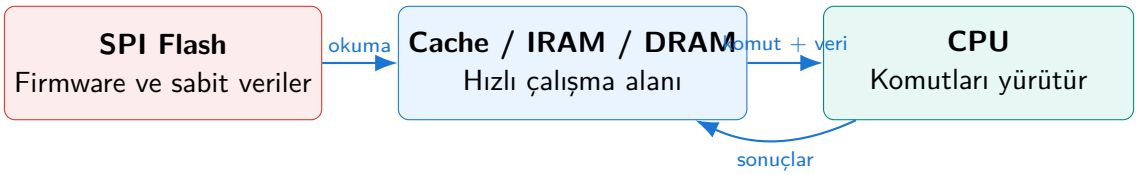
Her bölüm bu zincirin bir parçasını açıklar: sensörden veri almak, doğru pine yönlendirmek, bellekte işlemek ve sonucu başka bir cihaza veya aktüatöre göndermek.

2 Bellek, Kodun Çalışması ve Boot Süreci

2.1 Flash ve SRAM

SRAM ve Flash bellek nedir; ESP32'de nasıl birlikte çalışır?

Flash, elektrik kesildiğinde içeriğini koruyan kalıcı bellektir. Derlenmiş program, sabit veriler ve isteğe bağlı dosya sistemi burada tutulur. **SRAM** ise program çalışırken değişkenlerin, görev yığınlarının ve dinamik verilerin tutulduğu hızlı fakat geçici bellektir. Klasik ESP32-WROOM-32, 520 KB SRAM ve modüle bağlı tipik 4 MB SPI Flash ile gelir.[1][9]



Tablo 1: Flash ve SRAM'in pratik farkları

Özellik	Flash	SRAM
Bilgisayardaki benzeri	SSD / disk	RAM
Elektrik kesilince	Veri kalır	Veri silinir
Ne tutulur?	Firmware, sabit veri, dosyalar	Değişkenler, stack, heap, çalışma verisi
Hız	SRAM'e göre daha yavaş	Çok hızlı
Yazma ömrü	Sınırlı yazma/silme çevrimi	Çalışma boyunca sürekli kullanılabilir
Pratik risk	Çok sık kayıt Flash'ı yıpratabilir	Büyük diziler ve bellek sızıntıları sistemi kilitleyebilir

Kütüphane ve Çalışma Masası Analojisi

Flash, bütün kitapların saklandığı **kütüphane**; SRAM ise o anda kullandığınız belgelerin bulunduğu **çalışma masasıdır**. Masa daha hızlıdır fakat sınırlıdır ve güç kesilince temizlenir.

2.2 Stack, Heap ve Statik Bellek

- **Statik alan**: Global ve `static` değişkenler için derleme sırasında ayrılır.
- **Stack**: Fonksiyon çağrıları ve yerel değişkenler için kullanılır. FreeRTOS'ta her görevin kendi stack alanı vardır.[9]
- **Heap**: `malloc()`, `new`, dinamik `String` veya sonradan oluşturulan nesneler için kullanılır.

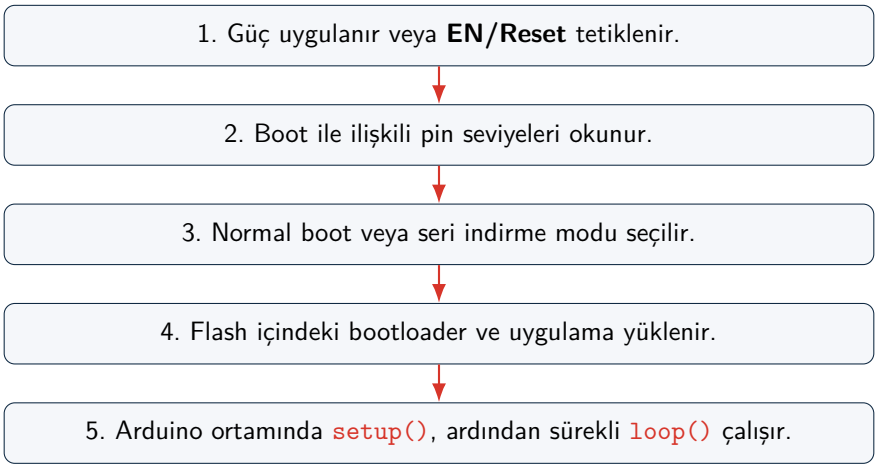
Bellek Parçalanması

Uzun süre çalışan projelerde sürekli büyüyüp küçülen dinamik `String` işlemleri ve sık `new/delete` kullanımı heap parçalanmasına yol açabilir. Sabit boyutlu tamponlar ve kontrollü veri yapıları daha öngörülebilir davranır.

2.3 Boot, Reset ve Upload

ESP32 açıldığında hangi sırayla çalışır?

Güç verildiğinde çip, strapping pinlerinin seviyelerini örnekler; boot modunu belirler; ROM içindeki ilk yükleyiciyi çalıştırır; Flash'taki bootloader ve uygulamayı doğrular; gerekli bölümleri hazırlar ve kullanıcı programını başlatır.[1]



Tablo 2: Kart üzerindeki temel düğmeler

Düğme / Pin	İşlev	Ne zaman kullanılır?
EN / RESET	Çipi yeniden başlatır	Program kilitlendiğinde veya yeni başlangıç gerektiğinde
BOOT / GPIO0	Reset anında LOW tutulursa indirme moduna girmeye yardım eder	Otomatik upload çalışmadığında elle yükleme için
USB-UART	Bilgisayar ile seri veri ve program yükleme köprüsüdür	Upload ve Serial Monitor

Elle Upload Sırası

Otomatik yükleme başarısızsa: **BOOT'a basılı tutun** → yüklemeyi başlatın → "Connecting..." görülünce gerekirse **EN'e kısa basın** → bağlantı kurulunca BOOT'u bırakın. Kart modeline göre bu adımlar değişebilir.

Bölüm Özeti

- Kod ve kalıcı ayarlar Flash'ta, çalışma verileri SRAM'dedir.
- Reset, Flash'taki programı silmez; yalnızca sistemi yeniden başlatır.
- GPIO0'ın reset anındaki seviyesi normal boot ile indirme modu arasındaki seçimi etkiler.

3 GPIO Mantığı ve Güvenli Pin Seçimi

3.1 GPIO Giriş ve Çıkış Mantığı

GPIO pinleri nasıl çalışır?

GPIO, "genel amaçlı giriş/çıkış" anlamına gelir. Bir pin yazılımla **giriş** yapıldığında dışarıdaki lojik seviyeyi okur; **çıkış** yapıldığında LOW veya HIGH seviyesi üretir. Klasik ESP32, 3.3 V lojikle çalışır. GPIO pinlerini 5 V lojik kabul eden pinler gibi değerlendirmemek gerekir.[1][2]

Tablo 3: Giriş ve çıkış modları

Mod	ESP32 ne yapar?	Tipik örnek	Kritik nokta
INPUT	Pin seviyesini okur	Buton, PIR, dijital sensör	Pin boşta kalırsa kararsız olabilir
INPUT_PULLUP	Dahili pull-up ile giriş okur	Butonun GND'ye bağlanması	GPIO34–39'da dahili pull-up/down yoktur
OUTPUT	LOW/HIGH üretir	Küçük LED, kontrol sinyali	Büyük yük doğrudan sürülmez
PWM / çevre birimi	Pine özel sinyal yönlendirilir	LED parlaklığı, UART, SPI	Pin kısıtları yine geçerlidir

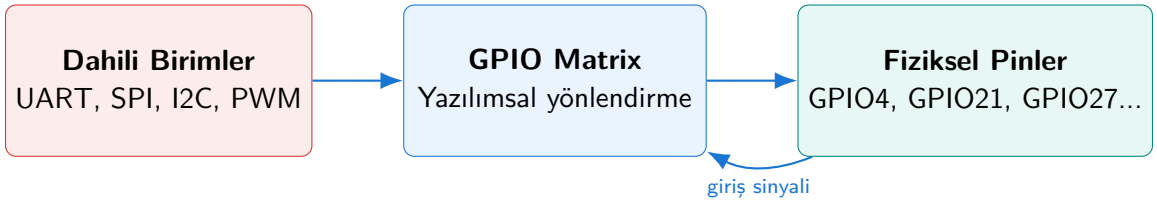
5 V ve Yük Sürme

- ESP32 GPIO girişine doğrudan 5 V uygulamayın. Gerekirse gerilim bölücü veya seviye dönüştürücü kullanın.
- Motor, röle, şerit LED veya yüksek güçlü LED'i GPIO'dan doğrudan beslemeyin. Transistör/MOSFET ve uygun güç kaynağı kullanın.
- Harici güç kaynağı kullanıyorsanız, haberleşme için çoğu durumda **ortak GND** gereklidir.

3.2 GPIO Matrix Nedir?

GPIO Matrix yaklaşımı ne sağlar?

GPIO Matrix, UART, I2C, SPI, PWM gibi dahili çevre birimi sinyallerini birçok fiziksel GPIO pinine yazılımla yönlendirebilen dijital anahtarlama katmanıdır. Bu sayede varsayılan pinler dışında alternatif pinler seçilebilir. Ancak fiziksel sınırlamalar ortadan kalkmaz: yalnızca giriş pinleri çıkış üretemez ve Flash/PSRAM için ayrılmış pinler serbestleşmez.[2]



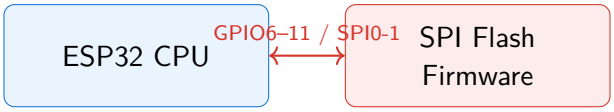
Kısa İpucu

Esnek pin ataması, "her pin her şeyi yapar" demek değildir. Pin seçiminde sırasıyla **fiziksel yetenek**, **boot etkisi**, **Flash/PSRAM bağlantısı**, **ADC birimi** ve **kart üzerindeki başka devreler** kontrol edilmelidir.

3.3 GPIO 6–11 Neden Kullanılmamalı?

GPIO 6–11 neden sensör, buton veya LED için kullanılmaz?

Tipik ESP32-WROOM geliştirme kartlarında GPIO6–11, modül üzerindeki SPI Flash ile haberleşmede kullanılır. İşlemci kodu ve veriyi bu bellekten aldığı için bu hatlara harici devre bağlamak boot hatalarına, çökme döngülerine veya programın hiç başlamamasına neden olabilir.[2]



Harici sensör bağlamak bu kritik veri yoluna müdahale eder.

Kritik Uyarı

GPIO6–11'i yalnızca "boş görünen pin numaraları" olarak değerlendirmeyin. Birçok geliştirme kartında bu pinler dışarı çıkarılmasa bile modül içinde Flash'a bağlıdır. Ayrıca bazı WROVER/PSRAM'li modüllerde GPIO16–17 de bellek bağlantıları için kullanılabilir.[2]

3.4 Pratik Pin Seçim Tablosu

Tablo 4: Klasik 38 pinli ESP32 DevKit için pratik pin grupları

Durum	Pinler	Açıklama
Genelde rahat	13, 14, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33	Genel dijital I/O için sık tercih edilir. 25–26 DAC, 32–33 ADC1 özelliklerine de sahiptir.
Koşullu rahat	16, 17	WROOM tabanlı birçok kartta kullanılabilir; PSRAM'li modüllerde ayrılmış olabilir.
Sadece giriş	34, 35, 36(VP), 39(VN)	Çıkış veremez; dahili pull-up/pull-down yoktur. Analog sensörler için değerlidir.
Dikkat	0, 2, 5, 12, 15	Boot/strapping davranışını etkileyebilir. Harici devrenin reset anındaki seviyesi önemlidir.
Paylaşımlı	1(TX0), 3(RX0)	USB seri haberleşme, upload ve debug ile çakışabilir.
ADC2	0, 2, 4, 12–15, 25–27	Wi-Fi aktifken klasik ESP32'de ADC2 kullanımı kısıtlıdır; analog okuma için ADC1 tercih edilir.[3]
Kullanma	6–11	Modülün SPI Flash bağlantısıdır.

Strapping Pin Mantığı

Strapping pinleri yalnızca reset/boot anında örneklenir. Program başladıktan sonra bazıları normal GPIO olarak kullanılabilir; ancak bağlı sensör veya direnç boot anında yanlış seviyeyi zorlarsa kart açılmayabilir. Özellikle GPIO0, GPIO2, GPIO5, GPIO12 ve GPIO15 için kart ve modül dokümanına bakın.[1][2]

3.5 Pin Seçme Algoritması

Bir Sensör İçin Pin Seçerken

1. Sensörün arayüzünü belirleyin: dijital, analog, I2C, SPI veya UART.
2. Pin çıkış verecek mi? Verecekse GPIO34–39'u eleyin.
3. Wi-Fi kullanacak mısınız? Analog girişte ADC1 pinlerini önceleyin.
4. GPIO6–11'i ve kartınızdaki Flash/PSRAM pinlerini eleyin.
5. Boot sırasında sensörün pini HIGH/LOW zorlayıp zorlamadığını kontrol edin.
6. UART0 kullanıyorsanız Serial Monitor ve upload çakışmasını düşünün.
7. Son olarak kart üzerindeki LED, buton veya USB devresinin aynı pine bağlı olup olmadığını kontrol edin.

4 Dijital ve Analog Dünya: ADC, DAC ve PWM

4.1 Üç Kavramın Farkı

ADC, DAC ve PWM arasındaki temel fark nedir?

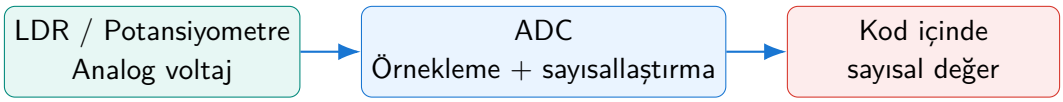
ADC, dışarıdan gelen analog voltajı sayısal değere dönüştürür. **DAC**, sayısal değeri gerçek analog voltaja dönüştürür. **PWM** ise pini çok hızlı açıp kapatarak ortalama güç etkisi oluşturur; gerçek sabit analog voltaj üretmek zorunda değildir.

Tablo 5: ADC, DAC ve PWM karşılaştırması

Özellik	ADC	DAC	PWM
Yön	Dış dünya → ESP32	ESP32 → dış dünya	ESP32 → anahtarlanan sinyal
Temel amaç	Voltaj ölçmek	Gerçek analog seviye üretmek	Güç/parlaklık/hız kontrolü
Klasik ESP32 örneği	GPIO32–39 (ADC1)	GPIO25 ve GPIO26	LEDC ile birçok çıkış pini
Örnek kullanım	LDR, potansiyometre	Referans voltajı, basit dalga	LED dimleme, fan kontrolü
Çıktı/girdi biçimi	Sayısal ölçüm	Analog voltaj	Darbe dizisi

4.2 ADC Nasıl Çalışır?

Klasik ESP32'de iki adet 12-bit SAR ADC birimi bulunur. ADC1, GPIO32–39; ADC2 ise GPIO0, 2, 4, 12–15 ve 25–27 üzerindedir. Wi-Fi sürücüsü ADC2 kaynaklarını kullandığından Wi-Fi açık projelerde analog sensör için ADC1 daha güvenilir seçimdir.[3]



ADC Değeri Her Zaman "Tam 0–3.3 V" Değildir

ADC ölçüm aralığı seçilen attenuation ayarına, çip farklarına ve kalibrasyona bağlıdır. Ham değer ile gerçek voltaj arasında tam doğrusal ilişki varsaymak hassas ölçümlerde hata doğurur. Gerçek bir ölçüm gerekiyorsa kalibrasyon, ortalama alma ve referans multimetre karşılaştırması yapın.

Listing 1: Basit analog okuma örneği

```
const int sensorPin = 34; // ADC1, yalnizca giris

void setup() {
  Serial.begin(115200);
}

void loop() {
  int hamDeger = analogRead(sensorPin);
  Serial.println(hamDeger);
  delay(100);
}
```

4.3 DAC Nasıl Çalışır?

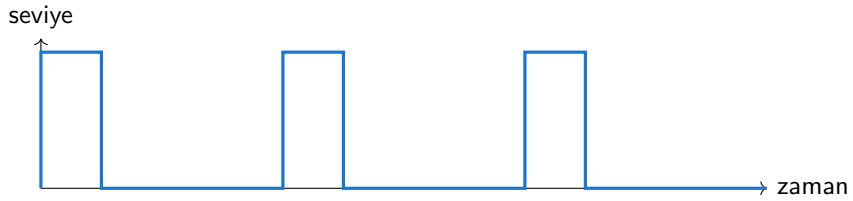
Klasik ESP32'de iki adet 8-bit DAC kanalı GPIO25 ve GPIO26 üzerindedir. `dacWrite(pin, deger)` çağrısında değer 0–255 aralığındadır. Bu özellik tüm ESP32 ailesinde bulunmaz; örneğin kart varyantınızı kontrol etmelisiniz.[5]

Kısa İpucu

DAC pinini küçük bir hoparlöre doğrudan yüksek güç vermek için kullanmayın. Ses veya yük sürme uygulamalarında uygun yükselteç, filtre ve koruma devresi gerekir.

4.4 PWM Neden Analog Gibi Görünür?

PWM, bir periyot içinde sinyalin açık kaldığı süreyi değiştirir. **Duty cycle** arttıkça LED daha parlak, motor daha hızlı veya ısıtıcı daha güçlü çalışıyor gibi görünür.

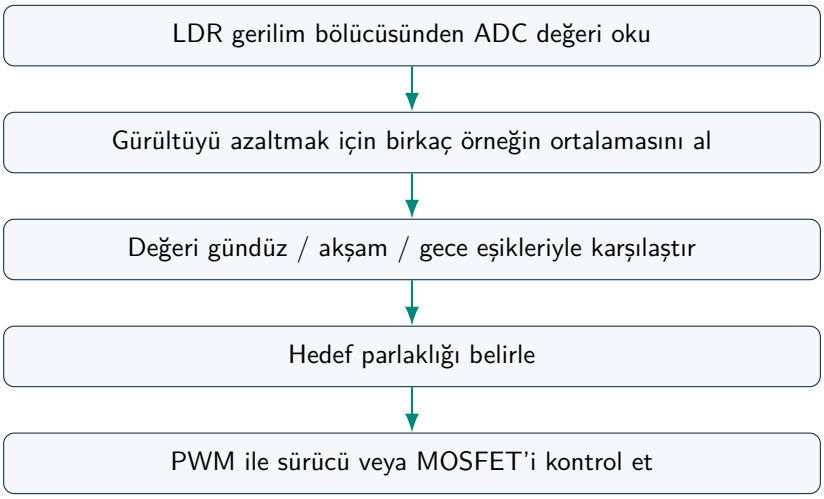


Düşük duty cycle ⇒ düşük ortalama güç

LED Parlaklığı

Beyaz LED'in parlaklığını artırmak için yalnızca direnci küçültmek doğru yaklaşım değildir. LED'in ileri yön gerilimi, akım sınırı ve GPIO'nun sürme kapasitesi dikkate alınmalıdır. Güçlü LED veya şerit LED için MOSFET üzerinden PWM kullanın.

4.5 LDR ile Otomatik Aydınlatma Mantığı



Bölüm Özeti

- Sensör voltajını okumak için ADC kullanılır.
- Gerçek analog voltaj üretmek için DAC kullanılır.
- LED parlaklığı ve motor hızı gibi güç kontrolünde çoğunlukla PWM kullanılır.
- Wi-Fi açıkken analog sensörü mümkünse ADC1 pinine bağlayın.

5 Kablolu Haberleşme: I2C, SPI ve UART

5.1 Genel Karşılaştırma

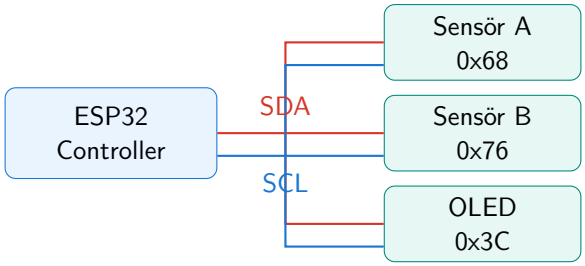
Tablo 6: I2C, SPI ve UART'ın pratik karşılaştırması

Özellik	I2C	SPI	UART
Saat hattı	Var	Var	Yok
Temel hatlar	SDA, SCL	SCLK, MOSI, MISO, CS	TX, RX
Çoklu cihaz	Adres ile kolay	Her cihaz için genellikle ayrı CS	Genellikle birebir
Hız	Orta	Yüksek	Baud rate'e bağlı
Kablo sayısı	Az	Daha fazla	Az
Tipik kullanım	Sensör, RTC, OLED	Ekran, SD kart, hızlı ADC	GPS, seri modül, debug

5.2 I2C

I2C nasıl iki kabloyla birden fazla sensörü çalıştırır?

I2C, veri için **SDA**, zamanlama için **SCL** hattını kullanır. Her cihazın bir adresi vardır; denetleyici hangi adresle konuşacağını bildirir ve yalnızca o cihaz cevap verir. Hatlar open-drain yapıda olduğundan SDA ve SCL için pull-up dirençleri gerekir. Birçok sensör modülünde bu dirençler kart üzerinde hazır bulunur.[4]



Klasik ESP32 Arduino kullanımında varsayılan pinler çoğunlukla SDA=GPIO21 ve SCL=GPIO22'dir; GPIO Matrix sayesinde başka uygun pinler seçilebilir.

Listing 2: I2C'yi varsayılan pinlerle başlatma

```
#include <Wire.h>

void setup() {
  Wire.begin(21, 22); // SDA, SCL
}
```

Aynı Adresli İki Sensör

İki I2C cihazı aynı sabit adrese sahipse aynı hatta doğrudan birlikte çalışmayabilir. Sensörün adres seçim pini, ikinci I2C veri yolu veya TCA9548A gibi I2C multiplexer kullanılabilir.

5.3 SPI

SPI neden hızlıdır ve neden daha çok pin kullanır?

SPI'da saat, gönderme, alma ve cihaz seçme hatları ayrıdır. Denetleyici, konuşacağı cihazın **CS** pinini etkinleştirir. Ayrı veri yönleri ve daha az protokol yükü yüksek hız sağlar; fakat her ek cihaz için çoğunlukla yeni bir CS pini gerekir.

Flash SPI ile Kullanıcı SPI'sını Karıştırmayın

GPIO6–11 üzerindeki SPI hatları modül Flash'ı içindir. Kullanıcı uygulamalarında tipik VSPI örneği SCLK=18, MISO=19, MOSI=23 ve CS=5'tir. Ancak GPIO5 bir strapping pini olduğundan bağlı cihazın boot anındaki davranışı kontrol edilmelidir.

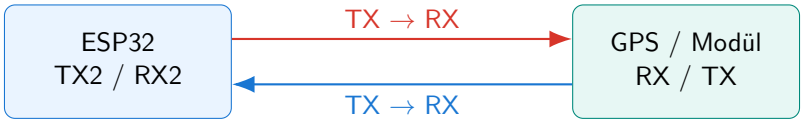
Tablo 7: SPI hatlarının görevi

Hat	Açılım	Görev
SCLK / CLK	Serial Clock	Denetleyicinin ürettiği saat
MOSI	Controller Out, Peripheral In	ESP32'den çevre birimine veri
MISO	Controller In, Peripheral Out	Çevre biriminden ESP32'ye veri
CS / SS	Chip Select	Hangi çevre biriminin aktif olduğunu seçer

5.4 UART

UART neden saat hattı olmadan çalışabilir?

UART asenkron haberleşir. İki taraf önceden aynı baud rate, veri biti, parity ve stop biti ayarlarını kabul eder. Gönderici ve alıcı, bit sürelerini bu ortak ayara göre yorumlar. TX ve RX çapraz bağlanır; çoğu devrede GND de ortaklanır.



Listing 3: UART2 başlatma örneği

```
HardwareSerial cihazSeri(2);

void setup() {
  cihazSeri.begin(9600, SERIAL_8N1, 16, 17); // RX, TX
}
```

UART0 çoğu kartta GPIO1/TX0 ve GPIO3/RX0 üzerinden USB-UART bağlantısına gider; program yükleme ve Serial Monitor için kullanılır. Harici modül için UART2'yi tercih etmek debug sürecini kolaylaştırır.

5.5 Hangi Protokolü Seçmeliyim?

Hızlı Karar

- Çok sayıda düşük/orta hızlı sensör, az kablo: **I2C**
- Ekran, SD kart, yüksek veri hızı: **SPI**
- GPS, modem, başka mikrodenetleyici veya basit seri akış: **UART**
- Sensör yalnızca voltaj üretiyorsa: **ADC**
- LED parlaklığı veya motor gücü ayarlanacaksa: **PWM**

6 Kablosuz Haberleşme: Wi-Fi, Bluetooth ve ESP-NOW

6.1 Wi-Fi 802.11 b/g/n

ESP32'de 802.11 b/g/n neyi ifade eder?

Bu ifadeler cihazın desteklediği 2.4 GHz Wi-Fi standartlarını gösterir. Klasik ESP32, 802.11b/g/n ağlarına bağlanabilir; 5 GHz Wi-Fi'a bağlanamaz. İstemci (STA), erişim noktası (SoftAP) veya ikisini birlikte kullanma senaryoları desteklenir.[1][8]

Wi-Fi, IP tabanlı haberleşme gerektiğinde uygundur: MQTT broker, web sunucusu, REST API, WebSocket veya internet erişimi gibi.

6.2 Bluetooth Classic ve BLE

Tablo 8: Bluetooth Classic ve BLE'nin pratik farkı

Özellik	Bluetooth Classic	Bluetooth Low Energy (BLE)
Ana amaç	Sürekli veri veya ses profilleri	Küçük veri paketleri, düşük güç
Tipik örnek	SPP seri bağlantı, ses	Sensör, beacon, mobil uygulama kontrolü
Veri modeli	Profil/kanal tabanlı	GATT servis ve characteristic yapısı
Güç yaklaşımı	Daha uzun aktif bağlantı	Kısa uyanma ve seyrek paketler
Klasik ESP32 desteği	Var	Var

Klasik ESP32'de Bluedroid yığını Bluetooth Classic ve BLE'yi; NimBLE ise daha küçük bellek ayak iziyle yalnız BLE'yi destekleyen seçenek olarak kullanılabilir.[6]

6.3 Zaman Paylaşımı ve RF Coexistence

Wi-Fi ve Bluetooth aynı anda nasıl çalışıyor?

Klasik ESP32'de Wi-Fi ve Bluetooth aynı 2.4 GHz RF kaynağını paylaşır. Yazılımda ikisi de aktif olabilir; ancak radyo aynı anda yalnız bir işlemin gönderim/alımını yapabilir. Coexistence mekanizması, radyo zamanını çok kısa dilimlere ayırır ve önceliklere göre Wi-Fi ile Bluetooth arasında paylaştırır.[7]



Tek radyo, hızlı zaman dilimleriyle paylaşılır

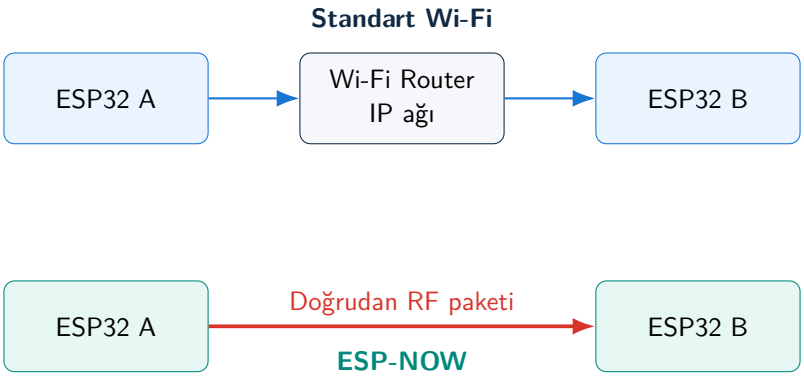
Mutfak Analojisi

Tek aşçı iki yemeği de pişirebilir; fakat aynı anda iki tencereye yüzde yüz dikkat veremez. İşleri hızlıca sıraya koyar. Wi-Fi ve Bluetooth coexistence mantığı da buna benzer.

6.4 ESP-NOW

ESP-NOW nedir ve standart Wi-Fi'dan farkı nedir?

ESP-NOW, Espressif'in bağlantısız, cihazdan cihaza haberleşme protokolüdür. Veri, Wi-Fi'ın fiziksel/MAC özellikleri kullanılarak gönderilir; fakat cihazların bir router'a bağlanması, IP alması veya TCP bağlantısı kurması gerekmez. Alıcılar genellikle MAC adresiyle eşleştirilir.[6]



Tablo 9: ESP-NOW'un önemli özellikleri ve pratik anlamı

Özellik	Pratik anlamı
Router gerektirmez	Saha düğümleri yerel olarak doğrudan haberleşebilir
IP/TCP zorunlu değildir	Daha az protokol yükü ve hızlı başlatma
MAC tabanlı peer yapısı	Hedef cihaz önceden tanımlanabilir veya broadcast kullanılabilir
Aynı Wi-Fi kanalı	Gönderici ve alıcı kanal uyumu sağlamalıdır
Teslim callback'i	MAC katmanı başarısını gösterir; uygulamanın veriyi işlediğini garanti etmez
Güvenlik	CCMP tabanlı şifreleme seçenekleri bulunur
Paket boyutu	ESP-NOW v1.0 için 250 bayt; güncel v2.0 cihazlarda 1470 bayta kadar destek vardır. Uyumluluk için küçük paketler tercih edilir.[6]

"Gönderim Başarılı" Her Zaman "Uygulama Aldı" Demek Değildir

Gönderim callback'inin başarılı olması, paketin MAC katmanında ulaştığını gösterir. Kritik sistemlerde paket numarası, zaman aşımı, alıcıdan ACK cevabı ve yeniden deneme mantığı ekleyin. Callback içinde uzun işlem yapmak yerine veriyi kuyruğa aktarın.

ESP-NOW Güvenilirlik Kontrol Listesi

1. Her iki cihazın Wi-Fi kanalını eşitleyin.
2. Gönderen ve alıcı veri yapılarının alan sırası ve boyutunu aynı tutun.
3. Pakete `nodeId`, `sequence`, `timestamp` ve veri sürümü ekleyin.
4. Alıcıda tekrar eden paketleri ayıklayın.

5. Kritik komutlarda uygulama seviyesinde ACK ve yeniden deneme kullanın.
6. Callback içinde yalnızca hızlı kopyalama/kuyruklama yapın.

6.5 Hangi Kablosuz Yöntem Seçilmeli?

Tablo 10: Kablosuz yöntem seçim rehberi

İhtiyaç	Wi-Fi	BLE	ESP-NOW
İnternet / MQTT / web	✅ Çok uygun	❌	❌ Doğrudan değil
Telefonla düşük güçlü kontrol	Uygun	✅ Çok uygun	Telefon desteği yok denecek kadar sınırlı
ESP32'ler arası düşük gecikme	Uygun	Uygun	✅ Çok uygun
Router olmadan saha ağı	SoftAP ile mümkün	Yakın mesafe	✅ Doğrudan
Yüksek veri akışı	✅	Sınırlı	Küçük/orta paketler
Düşük güç sensörü	Orta	✅	✅

Bölüm Özeti

- Backend, MQTT veya internet varsa Wi-Fi doğal seçimdir.
- Telefon ile düşük güçlü yakın alan bağlantısında BLE güçlüdür.
- ESP32 düğümleri arasında routersız, hızlı ve küçük paketli iletişimde ESP-NOW avantajlıdır.
- Wi-Fi ve ESP-NOW aynı radyoyu kullandığı için kanal planı özellikle gateway tasarımında önemlidir.

7 Sensörlerle Çalışma Rehberi

7.1 Sensörü Bağlamadan Önce Sorulacak Sorular

10 Adımlık Sensör Entegrasyon Akışı

1. **Parçanın tam modelini bulun.** Modül üzerindeki yazı ile sensör entegresini ayırt edin.
2. **Datasheet ve modül şemasını okuyun.** Besleme, lojik seviye ve pin görevlerini doğrulayın.
3. **Besleme gerilimini belirleyin.** 3.3 V mu, 5 V mu; modülde regülatör var mı?
4. **Çıkış seviyesini kontrol edin.** Sensör 5 V ile beslense bile veri çıkışı 5 V olabilir.
5. **Arayüzü belirleyin.** Analog, dijital, I2C, SPI veya UART.
6. **Uygun GPIO seçin.** Boot, ADC2/Wi-Fi ve giriş-only kısıtlarını kontrol edin.
7. **Ortak GND kurun.** Ayrı güç kaynaklarında referans noktası gereklidir.
8. **Önce en basit test kodunu çalıştırın.** Yalnız sensörü okuyun; sistemin tamamını aynı

anda kurmayın.

9. **Ham veriyi gözlemleyin.** Serial Monitor'da sınırlar, gürültü ve hata durumlarını görün.
10. **Kalibrasyon ve hata yönetimi ekleyin.** Ortalama, eşik, timeout ve bağlantı kontrolü uygulayın.

7.2 Modül ile Çıplak Sensör Farkı

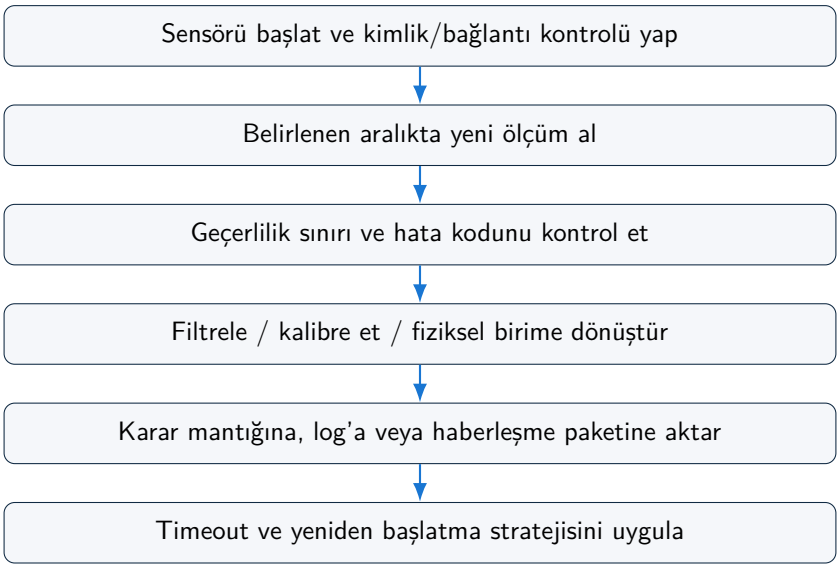
Tablo 11: Çıplak sensör ve hazır modül farkı

Konu	Çıplak sensör/entegre	Hazır modül kartı
Besleme	Datasheet'e doğrudan bağlı	Regülatör içerebilir
Pull-up / direnç	Sizin eklemeniz gerekebilir	Çoğu zaman kart üzerinde bulunur
Seviye dönüştürme	Genellikle yok	Bazı modüllerde bulunabilir
Pin isimleri	Entegre pinleri	VCC, GND, SCL, SDA, OUT gibi kolaylaştırılmış
Risk	Daha fazla tasarım bilgisi gerekir	Aynı isimli farklı üretici kartları değişebilir

Dikkat

"Modül 5 V ile çalışıyor" ifadesi, ESP32 veri pinine 5 V uygulanmasının güvenli olduğu anlamına gelmez. Kartın regülatör ve seviye dönüştürücü devresini şemadan doğrulayın.

7.3 Veri Okuma Döngüsü Nasıl Tasarlanmalı?



Non-blocking Tasarım

Uzun `delay()` çağrıları, kablosuz haberleşme ve birden fazla sensörün aynı anda yönetilmesini zorlaştırır. Periyodik işlerde `millis()` tabanlı zamanlama veya FreeRTOS görevleri tercih edilebilir.

7.4 Örnek: I2C Basınç Sensörü + ESP-NOW

Bu tür bir projede görevler şu şekilde ayrılabilir:

1. I2C ile sensör kimliğini ve kalibrasyon verilerini oku.
2. Belirli aralıkla sıcaklık/basınç ölçümü al.
3. Sonuçları fiziksel birime dönüştür.
4. Paket yapısına düğüm kimliği ve sıra numarası ekle.
5. ESP-NOW gönderim callback'inde yalnız sonucu kaydet veya kuyruğa aktar.
6. Alıcıda paket boyutu, sürüm ve sıra numarasını doğrula.

Kısa İpucu

Sensör okuması başarılı olduğu hâlde karşı tarafa eski veri gidiyorsa, paket kopyalama zamanı, callback kullanımı ve veri yapısının yaşam süresini kontrol edin.

8 Sık Sorulan Sorular ve Arıza Giderme

8.1 Kart Açılmıyor veya Boot Loop'a Giriyor

Neleri sırayla kontrol etmeliyim?

1. Harici devreyi tamamen söküp kartı yalnız USB ile çalıştırın.
2. 5 V'un yanlışlıkla 3V3/GPIO pinine uygulanmadığını kontrol edin.
3. GPIO0, 2, 5, 12 ve 15'e bağlı devrelerin boot seviyelerini inceleyin.
4. GPIO6–11'e bağlantı olmadığını doğrulayın.
5. Güç kaynağı ve USB kablosunun yeterli/sağlam olduğundan emin olun.
6. Serial Monitor'da reset nedeni ve boot mesajını okuyun.

8.2 Kod Yüklenmiyor

"Connecting..." aşamasında kalıyorsa ne yapmalıyım?

Doğru kart ve COM portunu seçin; veri taşıyan USB kablosu kullanın; Serial Monitor'u kapatın; BOOT düğmesini kullanarak elle indirme moduna girin; GPIO0/RX0/TX0'a bağlı harici devreleri geçici olarak sökün.

8.3 Wi-Fi Açılınca Analog Değer Bozuluyor

Neden oluyor?

Sensör ADC2 pinlerinden birine bağlı olabilir. Klasik ESP32'de Wi-Fi sürücüsü ADC2 ile kaynak paylaşır. Sensörü GPIO32–39 arasındaki uygun ADC1 pinine taşıyın.[3]

8.4 I2C Sensörü Bulunamıyor

En yaygın nedenler nelerdir?

- SDA ve SCL ters bağlanmıştır.
- Sensör adresi beklenenden farklıdır.
- Pull-up dirençleri yoktur veya yanlış gerilime çekilmiştir.
- Modül doğru beslenmiyordur ya da ortak GND yoktur.
- Kodda seçilen SDA/SCL pinleri fiziksel bağlantıyla uyuşmuyordur.
- Aynı hatta aynı adresli iki cihaz vardır.

Önce bir I2C scanner çalıştırarak hatta cevap veren adresleri görün.

8.5 ESP-NOW Paketleri Bazen Gelmiyor

Neleri kontrol etmeliyim?

- Cihazların aynı Wi-Fi kanalında olduğunu doğrulayın.
- Anten çevresinde metal, breadboard kablosu veya kapalı kutu etkisini kontrol edin.
- Paketleri çok sık göndermeyin; önceki callback'in dönmesini bekleyin.
- Yapı boyutu ve sürümünü iki tarafta eşitleyin.
- Paket numarası, ACK ve yeniden deneme ekleyin.
- Wi-Fi bağlantılı gateway kanalının router tarafından değişmediğinden emin olun.

8.6 LED Çok Kısık Yanıyor

Direnci küçültmek yeterli mi?

Önce LED'in yönünü, ileri yön gerilimini, kullanılan GPIO'nun gerçekten OUTPUT/HIGH olduğunu ve bağlantının ortak GND içerdiğini kontrol edin. Direnç değeri LED akımını belirler; çok küçültmek GPIO'ya veya LED'e zarar verebilir. Daha yüksek parlaklık veya güç için transistör/MOSFET sürücü kullanın.

8.7 GPIO34'e LED Bağladım, Neden Yanmıyor?

GPIO34–39 yalnız giriş özelliğine sahiptir; çıkış sürücüsü yoktur. Bu pinleri analog/dijital sensör okumak için kullanın, LED veya UART TX/PWM için kullanmayın.[2]

8.8 Serial Monitor'da Anlamsız Karakterler Var

Kodda kullanılan `Serial.begin(...)` baud rate değeri ile Serial Monitor ayarı aynı olmalıdır. Reset anındaki ROM boot mesajları farklı baud rate ile görünebilir; uygulama başladıktan sonraki çıktıyı esas alın.

Arıza Giderme İlkesi

Sorunu küçültün: **güç** → **tek sensör** → **ham okuma** → **haberleşme** → **tam sistem**. Aynı anda beş değişkeni değiştirmek, hatanın kaynağını görünmez hâle getirir.

9 Hızlı Başvuru Sayfası

9.1 Bir Bakışta Pin Seçimi

Önce Düşünülebilecek Pinler

13, 14, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33
Not: Protokol, ADC/DAC ve boot gereksinimlerine göre tekrar kontrol edin.

Dikkatli Kullan

0, 2, 5, 12, 15: boot/strapping
1, 3: USB seri haberleşme
16, 17: PSRAM varyantı kontrolü

Sadece Giriş

34, 35, 36(VP), 39(VN)
Dahili pull-up/down yoktur.

Kullanma

GPIO6–11
Modül SPI Flash bağlantılarıdır.

9.2 Bir Bakışta Haberleşme Seçimi

İhtiyaç	İlk bakılacak teknoloji
Çok sensör, iki veri hattı	I2C
Yüksek hızlı ekran / SD kart	SPI
GPS / seri modül / debug	UART
Analog sensör	ADC1
LED parlaklığı / motor gücü	PWM + sürücü
Gerçek analog seviye	DAC (destekli ESP32 varyantı)
İnternet / MQTT / dashboard	Wi-Fi
Telefonla düşük güçlü bağlantı	BLE
ESP32'ler arası routersız iletişim	ESP-NOW

9.3 Proje Başlatma Kontrol Listesi

Masaya Oturmadan Önce

- ☐ Kart ve modül modeli doğrulandı
- ☐ Güç gerilimleri kontrol edildi
- ☐ Ortak GND planlandı
- ☐ Pin çakışmaları çıkarıldı
- ☐ Boot pinleri kontrol edildi
- ☐ ADC1/ADC2 seçimi yapıldı
- ☐ Sensör adresi/protokolü belirlendi
- ☐ Basit test kodu hazırlandı
- ☐ Serial log formatı belirlendi
- ☐ Timeout ve hata durumu düşünüldü
- ☐ Kablosuz kanal/peer planlandı
- ☐ Yük sürücüsü ve koruma eklendi

10 Yardımcı Kaynaklar ve Kaynakça

Bu rehberdeki teknik özellikler, klasik ESP32/ESP32-WROOM-32 kapsamındaki resmî Espressif belgeleri esas alınarak özetlenmiştir. Kart üreticisinin pin şeması ve kullandığınız Arduino-ESP32/ESP-IDF sürümünün belgeleri ayrıca kontrol edilmelidir.

[1] Espressif — ESP32-WROOM-32 Datasheet, v3.6.
Modül özellikleri, pin tanımları, boot yapılandırmaları, bellek ve elektriksel sınırlar.
documentation.espressif.com/esp32-wroom-32_datasheet_en.html

[2] Espressif — GPIO & RTC GPIO, ESP-IDF Programming Guide.
GPIO yetenekleri, input-only pinler, strapping pinleri, Flash/PSRAM notları.
docs.espressif.com/.../esp32/api-reference/peripherals/gpio.html

[3] Espressif — Analog to Digital Converter (ADC), ESP-IDF.
ADC1/ADC2 kanalları ve Wi-Fi ile ADC2 paylaşım kısıtları.
docs.espressif.com/.../peripherals/adc_oneshot.html

[4] Espressif — Arduino-ESP32 I2C API.
SDA/SCL, pull-up gereksinimi, controller/peripheral modları ve örnekler.
docs.espressif.com/projects/arduino-esp32/en/latest/api/i2c.html

[5] Espressif — Arduino-ESP32 DAC API.
Desteklenen çipler, GPIO25/26 ve 8-bit DAC kullanımı.
docs.espressif.com/projects/arduino-esp32/en/latest/api/dac.html

[6] Espressif — ESP-NOW ve Bluetooth API Belgeleri.
ESP-NOW paket yapısı, sürüm uyumluluğu, callback ve güvenlik; Bluetooth host seçenekleri.
ESP-NOW Programming Guide
Bluetooth API Reference

[7] Espressif — RF Coexistence.
Wi-Fi, Bluetooth/BLE ve ESP-NOW'un ortak 2.4 GHz radyo kaynağını zaman dilimleriyle paylaşması.
docs.espressif.com/.../esp32/api-guides/coexist.html

[8] Espressif — ESP32 Wi-Fi Driver Overview.
STA, SoftAP, 802.11b/g/n, protokol özellikleri ve uygulama yaklaşımı.
docs.espressif.com/.../wifi-driver/overview.html

[9] Espressif — Heap Memory Allocation ve Memory Types.
Stack, heap, DRAM, IRAM ve yetenek tabanlı bellek yönetimi.
docs.espressif.com/.../system/mem_alloc.html

Ek öğrenme kaynağı — Espressif örnek projeleri.
Arduino-ESP32 örnekleri ve ESP-IDF'nin resmî `examples` klasörü, her çevre birimini küçük bir proje ile denemek için en güvenilir başlangıç noktalarıdır.
github.com/espressif/arduino-esp32
github.com/espressif/esp-idf/examples

Öğrenmenin en hızlı yolu: tek bileşen, basit devre, ölçülebilir çıktı.